実験作業者メイン画面 UI設計書

メタデータ

- **画面名**: 実験作業者メイン画面
- **物理名**: ExperimentWorkerMenu.tsx
- **ファイルパス**: frontend/src/components/menus/ExperimentWorkerMenu.tsx
- **作成日**: 2026-01-21
- **最終更新日**: 2026-03-11
- **担当者**: Development Team
- **バージョン**: 1.1.0

---

1. 画面概要

1.1. 目的

実験作業者が実験データの記録、進捗確認、作業指示の確認を行うためのメイン画面です。プロジェクト選択後、3つのタブを切り替えることで各機能にアクセスします。

1.2. アクセス権限

- **ロール**: `experiment_worker`（実験作業者）
- **認証**: 必須

1.3. 主要機能

1. プロジェクト選択（APIから取得した一覧から選択）
2. タブ切り替え（進捗確認、指示確認、実験記録）
3. メソッド・タスク選択（指示確認タブ・実験記録タブのみ）
4. 承認依頼（進捗確認タブのみ、APIで送信）

1.4. 表示モード

モード	説明
通常モード	Header + SubHeader + コンテンツを表示（デフォルト）
オーバーレイモード（ `overlayMode=true` ）	Header/SubHeaderを非表示にしてコンテンツのみ表示。他画面への埋め込み用
読み取り専用モード（ `readOnly=true` ）	選択・承認依頼ボタンを disabled にする。参照目的での表示用

1.5. コンポーネントProps定義

```
interface ExperimentWorkerMenuProps {
  /** オーバーレイモード: trueにするとHeader/SubHeaderを非表示にしてコンテンツのみ表示 */
  overlayMode?: boolean;

  /** プロジェクト固定: 指定するとそのプロジェクトが選択された状態で固定され、選択コンボボックスが
disabled になる */
  fixedProjectId?: string;

  /** 読み取り専用: trueにすると選択ボタン・承認依頼ボタンを disabled にする */
  readOnly?: boolean;
}
```

Prop名	型	デフォルト値	説明
`overlayMode`	`boolean`	`false`	`true` で Header/SubHeader を非表示にする
`fixedProjectId`	`string`	`undefined`	プロジェクトを固定選択する（コンボボックスが disabled）
`readOnly`	`boolean`	`false`	`true` で選択・承認依頼ボタンを disabled にする

---

2. 画面レイアウト

2.1 画面イメージ



2.1. 全体構成



2.2. レイアウト定数

要素	高さ	説明
Header	64px	共通ヘッダー
SubHeader	48px	サブヘッダー（タブなし）
プロジェクト選択エリア	56px	プロジェクト選択・ボタン配置
タブバー	48px	タブ切り替え
コンテンツエリア	可変	タブごとのコンテンツ

合計固定高さ: 216px（Header + SubHeader + 選択エリア + タブバー）

---

3. プロジェクト選択エリア

3.1. UI構成

```
┌──────────────────────────────────────────────────────────┐
│ 📁 プロジェクト: [プロジェクト選択 ▼] [選択] [承認依頼]        │
└──────────────────────────────────────────────────────────┘
```

3.2. コンポーネント詳細

3.2.1. プロジェクト選択コンボボックス

項目	値
コンポーネント	`Select` (Ant Design)
幅	280px
サイズ	`middle`
プレースホルダー	"-"
アイコン	`FolderOutlined`
ラベル	"プロジェクト:"

表示形式:

```
{プロジェクト名} - {ステータス}
例: Sample Project - 作業中
```

プロジェクトステータス（数値）:

値	意味
`0`	計画中
`1`	タスク実行待ち
`2`	タスク実行中
`3`	承認待ち
`4`	完了
`5`	異常終了

ステータスラベルは `useCodeMaster('project_status')` フックから取得する。フロントエンドによるフィルタリングは行わず、APIから返ってきたすべてのプロジェクトを選択肢として表示す

る。

ドロップダウンオプション:

```
const projectStatusOptions = useCodeMaster('project_status');

options={[
  { value: '', label: '-' },
  ...projects.map((project) => {
    const statusLabel = projectStatusOptions.find(opt => opt.value ===
project.projectStatus)?.label
      ?? String(project.projectStatus);
    return {
      value: project.projectId,
      label: `${project.projectName} - ${statusLabel}`,
    };
  })
]}
```

3.2.2. 選択ボタン

項目	値
コンポーネント	`Button` (Ant Design)
タイプ	`primary`
サイズ	`large`
アイコン	`CheckOutlined`
テキスト	"選択"
無効化条件	なし（常に有効）

動作:

- クリック時: `ProjectSelectionModal` を開く
- テーブル形式でプロジェクト一覧を表示し、**プロジェクトのみ**を選択する
- あいまい検索・プロジェクト名フィルタ機能あり
- メソッド・タスクの選択はこのモーダルでは行わない（`MethodTaskSelectionModal` で別途選択）

3.2.3. 承認依頼ボタン

項目	値
コンポーネント	`Button` (Ant Design)
タイプ	`default`
サイズ	`large`
テキスト	"承認依頼"
表示条件	進捗確認タブ（`currentTab === '4'`）選択時のみ
無効化条件	プロジェクト未選択時（`!selectedProject

動作:

- クリック時: `handleApprovalRequest()` を実行
- `PATCH /api/v1/projects/{projectName}/approval-request` を呼び出す
- 送信中は `approvalLoading=true` になりボタンにローディング表示

3.3. レイアウト

```
<div style={{
  height: 56,
  display: 'flex',
  alignItems: 'center',
  justifyContent: 'flex-start',
  gap: token.marginXS,
  flexWrap: 'nowrap',
  backgroundColor: token.colorBgContainer,
  border: `1px solid ${token.colorBorder}`,
  borderLeft: 'none',
  borderRight: 'none',
  paddingLeft: token.paddingLG,
  paddingRight: token.paddingLG,
}}>
  <Space size="middle">
    <FolderOutlined style={{ fontSize: 16, color: token.colorPrimary }} />
    <Text strong>プロジェクト:</Text>
    {/* プロジェクト選択コンボボックス（fixedProjectId指定時は disabled） */}
    <Select
      loading={projectsLoading}
      disabled={!!fixedProjectId}
```

```
      {...}
    />
    {/* 選択ボタン（readOnly時は非表示） */}
    {!readOnly && (
      <Button type="primary" {...}>選択</Button>
    )}
    {/* 承認依頼ボタン（進捗確認タブのみ表示、readOnly時は非表示） */}
    {currentTab === '4' && !readOnly && (
      <Button loading={approvalLoading} {...}>承認依頼</Button>
    )}
  </Space>
</div>
```

3.4. 状態管理

状態	型	初期値	説明
`selectedProject`	`string`	`''`	選択中のプロジェクトID
`selectedProjectName`	`string`	`''`	選択中のプロジェクト名（表示用）
`projects`	`WorkerProject[]`	`[]`	APIから取得したプロジェクト一覧
`projectsLoading`	`boolean`	`false`	プロジェクト一覧取得中フラグ
`selectedMethodId`	`string | null`	`null`	選択中のメソッドID
`selectedTaskId`	`string`	`'new'`	選択中のタスクID
`dialogOpen`	`boolean`	`false`	プロジェクト選択モーダルの開閉状態
`approvalLoading`	`boolean`	`false`	承認依頼送信中フラグ

WorkerProject 型定義:

```
interface WorkerProject {
  projectId: string;       // プロジェクトID（UUID）
  projectName: string;     // プロジェクト名
  description: string;     // 説明
  projectStatus: number;   // ステータス（数値）
  useDeemedProgress?: boolean; // みなし進捗フラグ
  createdAt: string;       // 作成日時（ISO 8601）
  updatedAt: string;       // 更新日時（ISO 8601）
}
```

3.5. イベントハンドラー

プロジェクト変更時

```
const handleProjectChange = (value: string | undefined | null) => {
  const projectValue = value || '';
  // プロジェクト名も同時に更新（ProgressCheckTab等への受け渡し用）
  const project = projects.find(p => p.projectId === projectValue);
  const projectName = project?.projectName ?? '';

  originalHandleProjectChange(projectValue, projectName); // 第2引数にprojectNameを渡す
  setSelectedProjectName(projectName);

  // メソッドとタスクをリセット
  setSelectedMethodId(null);
  setSelectedTaskId('');
};
```

動作:

1. プロジェクトIDを更新
2. プロジェクト名を取得して `selectedProjectName` に保存
3. `useInstructions` の `handleProjectChange(projectId, projectName)` を呼び出す
4. メソッドを `null` にリセット
5. タスクを空文字列にリセット

---

4. タブ構成

4.1. タブ一覧

タブキー	ラベル	アイコン	コンポーネント	説明
`'4'`	進捗確認	`CheckSquareOutlined`	`ProgressCheckTab`	実験進捗の確認
`'2'`	指示確認	`FileTextOutlined`	`InstructionConfirmationTab`	作業指示の確認

`'3'`	実験記録	`FormOutlined`	`ExperimentRecordTab`	実験データの記録

4.2. タブバースタイル

```
<Tabs
  activeKey={currentTab}
  onChange={handleTabChange}
  items={tabItems}
  size="large"
  style={{
    marginBottom: 0,
  }}
  tabBarStyle={{
    padding: `0 ${token.paddingLG}px`,
    margin: 0,
    borderTop: `1px solid ${token.colorBorder}`,
    borderBottom: `1px solid ${token.colorBorder}`,
    borderLeft: 'none',
    borderRight: 'none',
    borderRadius: 0,
  }}
/>
```

4.3. タブアイテム定義

```
const tabItems = [
  {
    key: '4',
    label: t('menu.experiment_worker.tabs.progressCheck'),
    icon: <CheckSquareOutlined />,
    children: <ProgressCheckTab {...} />,
  },
  {
    key: '2',
    label: t('menu.experiment_worker.tabs.workInstruction'),
    icon: <FileTextOutlined />,
    children: <InstructionConfirmationTab {...} />,
  },
  {
    key: '3',
    label: t('menu.experiment_worker.tabs.experimentRecord'),
    icon: <FormOutlined />,
```

```
    children: <ExperimentRecordTab {...} />,
  },
];
```

4.4. タブ切り替え

```
const handleTabChange = (key: string) => {
  setCurrentTab(key);
};
```

動作:

- タブクリック時、`currentTab` 状態を更新

各タブコンポーネントに渡すprops:

Props	進捗確認タブ	指示確認タブ	実験記録タブ
`selectedProject`	✅	✅	✅
`selectedProjectName`	✅	-	-
`onProjectChange`	-	✅	✅
`selectedMethodId`	-	✅	✅
`onSelectMethod`	-	✅	✅
`selectedTaskId`	-	✅	✅
`onSelectTask`	-	✅	✅
`onTabChange`	✅	-	-
`onSelectTaskAndTab`	✅	-	-
`readOnly`	✅	-	-

---

5. 共通コンポーネント: メソッド・タスク選択エリア

注意: このエリアは**指示確認タブ**と**実験記録タブ**のみに表示されます。進捗確認タブには表示されません。

5.1. コンポーネント概要

- **物理名**: `MethodTaskSelector.tsx`
- **ファイルパス**: frontend/src/components/ExperimentWorker/MethodTaskSelector.tsx
- **用途**: メソッドとタスクの選択UI（指示確認タブと実験記録タブで共通利用）

5.2. UI構成

```
┌──────────────────────────────────────────────────────────┐
│ 🔀 メソッド: [メソッド選択 ▼]  ☑ タスク: [タスク選択 ▼] [選択] │
└──────────────────────────────────────────────────────────┘
```

5.3. コンポーネント詳細

5.3.1. メソッド選択

項目	値
コンポーネント	`Select` (Ant Design)
幅	200px
サイズ	`large`
プレースホルダー	"メソッドを選択してください"
アイコン	`PartitionOutlined`
ラベル	"メソッド:"
ラベル幅	80px

オプション:

```
options={[
  { label: '-', value: null },
  ...methods.map((method) => ({
    label: method.methodName,
    value: method.methodId,
```

```
  })),
]}
```

5.3.2. タスク選択

項目	値
コンポーネント	`Select` (Ant Design)
幅	200px
サイズ	`large`
プレースホルダー	"タスクを選択してください"
アイコン	`CheckSquareOutlined`
ラベル	"タスク:"
ラベル幅	80px

オプション:

```
options={[
  { label: '-', value: '' },
  ...tasks.map((task) => ({
    label: task.taskName,
    value: task.taskId,
  })),
]}
```

5.3.3. 選択ボタン

項目	値
コンポーネント	`Button` (Ant Design)
タイプ	`primary`
サイズ	`large`
アイコン	`CheckOutlined`
テキスト	"選択"
無効化条件	プロジェクト未選択時（ `!selectedProject` ）

動作:

- クリック時: `MethodTaskSelectionModal` を開く
- メソッドとタスクを同時に選択可能

5.4. レイアウト

```
<div style={{
  marginBottom: 8,
  height: 56,
  flexShrink: 0,
  display: 'flex',
  alignItems: 'center',
  paddingLeft: token.paddingLG,
  paddingRight: token.paddingLG,
  backgroundColor: token.colorBgContainer,
  border: `1px solid ${token.colorBorder}`,
  borderRadius: 0,
}}>
  <div style={{ display: 'flex', alignItems: 'center', width: '100%', gap: token.margin }}>
    {/* メソッド選択 */}
    <Space size="middle" style={{ minWidth: 300, maxWidth: 400 }}>
      <PartitionOutlined style={{ fontSize: 16, color: token.colorPrimary }} />
      <Text strong style={{ minWidth: 80 }}>メソッド:</Text>
      <Select {...} />
    </Space>

    {/* タスク選択 + 選択ボタン */}
    <Space size="middle" style={{ minWidth: 300, maxWidth: 400 }}>
      <CheckSquareOutlined style={{ fontSize: 16, color: token.colorPrimary }} />
      <Text strong style={{ minWidth: 80 }}>タスク:</Text>
      <Select {...} />
      <Button type="primary" {...}>選択</Button>
    </Space>
  </div>
</div>
```

5.5. Props定義

```
interface MethodTaskSelectorProps {
  // プロジェクト選択状態
```

```
  selectedProject: string;

  // メソッド関連
  selectedMethodId: string | null;
  onSelectMethod: (methodId: string | null) => void;
  methods: Array<{ methodId: string; methodName: string }>;

  // タスク関連
  selectedTaskId: string;
  onSelectTask: (taskId: string) => void;
  tasks: Array<{ taskId: string; taskName: string }>;

  // モーダル制御
  onOpenModal: () => void;

  // Disabled状態
  methodDisabled?: boolean;
  taskDisabled?: boolean;
}
```

5.6. 使用箇所

このコンポーネントは以下の2つのタブで使用されます：

1. **指示確認タブ**（`InstructionConfirmationTab.tsx`）
2. **実験記録タブ**（`ExperimentRecordTab.tsx`）

進捗確認タブには表示されません。

---

6. レスポンシブデザイン

6.1. ブレークポイント

ブレークポイント	画面幅	コンテナ最大幅	パディング
xs, sm	< 768px	100%	8px
md	768px+	768px	16px
lg	992px+	992px	16px
xl	1200px+	1200px	16px

xxl	1600px+	1400px	16px

6.2. レスポンシブコンテナ

```
const ResponsiveContainer = styled.div`
  max-width: 100%;
  width: 100%;
  margin: 0 auto;
  padding: 0 8px;

  @media (min-width: 768px) {
    max-width: 768px;
    padding: 0 16px;
  }

  @media (min-width: 992px) {
    max-width: 992px;
  }

  @media (min-width: 1200px) {
    max-width: 1200px;
  }

  @media (min-width: 1600px) {
    max-width: 1400px;
  }
`;
```

6.3. モバイル対応

xs, sm（< 768px）:

- プロジェクト選択エリアは縦並び（`flexDirection: 'column'`）
- メソッド・タスク選択エリアは縦並び
- ボタンは全幅（`width: '100%'`）

md以上（768px+）:

- プロジェクト選択エリアは横並び（`flexDirection: 'row'`）
- メソッド・タスク選択エリアは横並び

- ボタンは固定幅

---

7. 状態管理

7.1. ローカル状態

状態名	型	初期値	説明
`currentTab`	`string`	`'4'`	現在のタブキー（進捗確認）
`isClient`	`boolean`	`false`	クライアント側レンダリングフラグ
`selectedProjectName`	`string`	`''`	選択中のプロジェクト名（表示用）
`projects`	`WorkerProject[]`	`[]`	APIから取得したプロジェクト一覧
`projectsLoading`	`boolean`	`false`	プロジェクト一覧取得中フラグ
`selectedMethodId`	`string | null`	`null`	選択中のメソッドID
`selectedTaskId`	`string`	`'new'`	選択中のタスクID
`dialogOpen`	`boolean`	`false`	プロジェクト選択モーダルの開閉
`approvalLoading`	`boolean`	`false`	承認依頼送信中フラグ

7.2. カスタムフック

useInstructions

```
const {
  selectedProject,          // 選択中のプロジェクトID
  sortedInstructions,       // ソート済み指示リスト
  stats,                    // 統計情報
  handleProjectChange,      // プロジェクト変更ハンドラー（引数: projectId: string, projectName:
string）
} = useInstructions();
```

handleProjectChange のシグネチャ（useInstructions内部）：

```
// useInstructions.ts
const handleProjectChange = (projectId: string, projectName: string) => {
  setSelectedProject(projectId);
  // projectName は指示一覧取得時のフィルタリングに使用
};
```

useExpandState

```
const {
  expandedGroups,  // 展開中のグループID配列
  toggleGroup,     // グループの展開/折りたたみ
} = useExpandState();
```

---

8. イベントハンドラー

8.1. プロジェクト変更

```
const handleProjectChange = (value: string | undefined | null) => {
  const projectValue = value || '';
  // プロジェクト名も同時に更新
  const project = projects.find(p => p.projectId === projectValue);
  const projectName = project?.projectName ?? '';

  originalHandleProjectChange(projectValue, projectName); // 第2引数にprojectNameを渡す
  setSelectedProjectName(projectName);

  // メソッドとタスクをリセット
  setSelectedMethodId(null);
  setSelectedTaskId('');
};
```

動作:

1. プロジェクトIDを更新
2. プロジェクト名を `projects` 配列から検索して取得

3. `selectedProjectName` を更新
4. `useInstructions` の `handleProjectChange(projectId, projectName)` を呼び出す
5. メソッドIDを `null` にリセット
6. タスクIDを空文字列にリセット

8.2. メソッド選択

```
const handleSelectMethod = (methodId: string | null) => {
  // メソッドが実際に変更された場合のみタスクをリセット
  if (methodId !== selectedMethodId) {
    setSelectedTaskId('');
  }
  setSelectedMethodId(methodId);
};
```

動作:

1. メソッドが変更された場合、タスクをリセット
2. メソッドIDを更新

8.3. タスク選択

```
const handleSelectTask = (taskId: string) => {
  setSelectedTaskId(taskId);
};
```

8.4. タスク発行

```
const handleTaskCreated = (newTaskId: string) => {
  setSelectedTaskId(newTaskId);
};
```

8.5. タスク選択とタブ切り替え

```
const handleSelectTaskAndTab = (taskId: string, tabKey: string) => {
  // TODO: API化後、タスクIDからメソッドIDを取得するロジックを実装
  // 現在はモックデータを使用していないため、メソッドIDの取得は省略

  // タスクを選択
  setSelectedTaskId(taskId);
  // タブを切り替え
  setCurrentTab(tabKey);
};
```

用途: 進捗確認タブから実験記録タブへのタスク遷移

⚠️ **注意:** メソッドIDの取得ロジックは**未実装（TODO）**。API化後に、タスクIDからメソッドIDを逆引きするAPIまたはレスポンス構造を利用して実装予定。

8.6. 承認依頼

```
const handleApprovalRequest = async () => {
  if (!selectedProject || !selectedProjectData) return;

  setApprovalLoading(true);
  try {
    // projectName を使ってAPIを呼び出す（PATCH /{projectName}/approval-request）
    await requestApproval(selectedProjectData.projectName);
    message.success('承認依頼を送信しました');

    // プロジェクト一覧を再取得してステータスを更新
    const projectsResult = await getWorkerAccessibleProjects();
    if (projectsResult.success) {
      setProjects(projectsResult.data);
    }
  } catch (error) {
    message.error('承認依頼の送信に失敗しました');
  } finally {
    setApprovalLoading(false);
  }
};
```

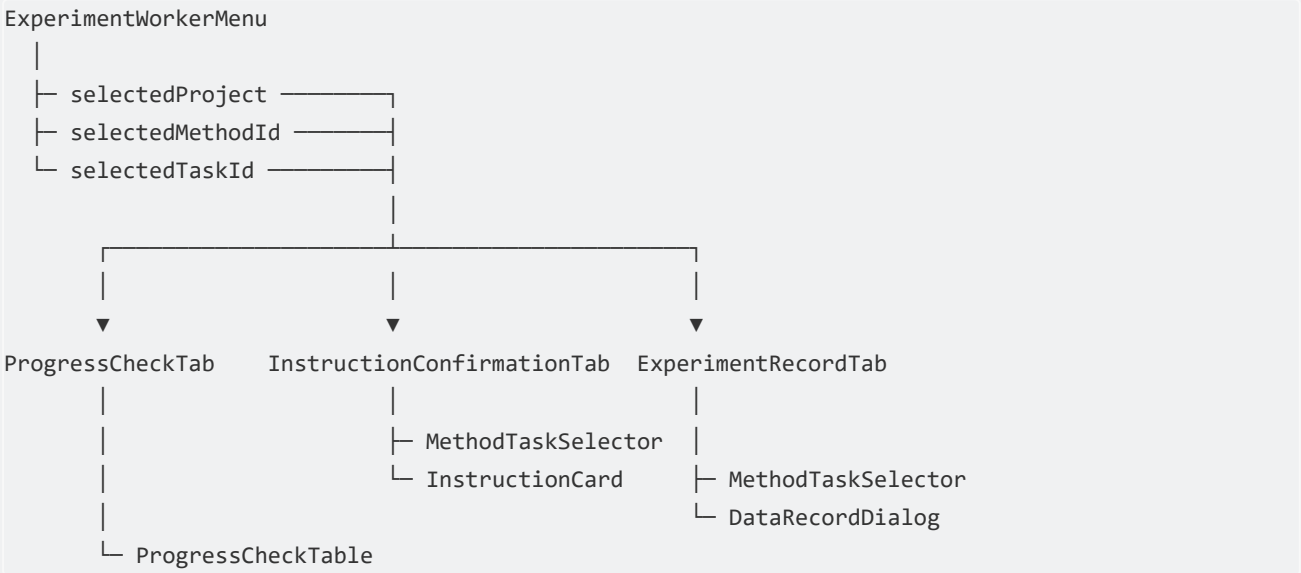
動作:

1. `approvalLoading` を `true` に設定（ボタンにローディング表示）
2. `requestApproval(projectName)` を呼び出し（`PATCH /api/v1/projects/{projectName}/approval-request`）
3. 成功時: 成功メッセージを表示し、プロジェクト一覧を再取得してステータスを反映
4. 失敗時: エラーメッセージを表示
5. `approvalLoading` を `false` に戻す

---

9. データフロー

9.1. 選択状態の伝播



9.2. 選択状態のリセット条件

トリガー	プロジェクト	メソッド	タスク
プロジェクト変更	✓ 変更	✓ リセット	✓ リセット
メソッド変更	-	✓ 変更	✓ リセット
タスク変更	-	-	✓ 変更

---

10. スタイリング

10.1. デザイントークン使用

```
const { token } = theme.useToken();

// 使用例
style={{
  backgroundColor: token.colorBgContainer,
  border: `1px solid ${token.colorBorder}`,
  paddingLeft: token.paddingLG,
  paddingRight: token.paddingLG,
  gap: token.margin,
  color: token.colorPrimary,
}}
```

10.2. 主要なトークン

トークン	用途	値（デフォルト）
`token.colorBgContainer`	コンテナ背景色	`#ffffff`
`token.colorBgLayout`	レイアウト背景色	`#f5f5f5`
`token.colorBorder`	ボーダー色	`#d9d9d9`
`token.colorPrimary`	プライマリ色	`#1890ff`
`token.paddingLG`	大パディング	`24px`
`token.margin`	マージン	`16px`
`token.marginXS`	極小マージン	`4px`

11. アクセシビリティ

11.1. キーボード操作

操作	動作
`Tab`	フォーカス移動
`Enter`	ボタンクリック、Select開く
`Space`	ボタンクリック
`↑` / `↓`	Selectオプション選択
`Esc`	Selectを閉じる

11.2. スクリーンリーダー対応

- すべてのボタンに `aria-label` を自動付与（Ant Design標準）
- Selectに `placeholder` を設定
- アイコンに `aria-hidden="true"` を付与（装飾的アイコン）

---

12. パフォーマンス最適化

12.1. メモ化

```
// 選択されたメソッドを取得（計算コストが高い場合はメモ化）
const selectedMethod = useMemo(() => {
  if (!selectedProject || !selectedMethodId) return null;
  return projectsData[selectedProject]?.methods?.find(
    (m) => m.methodId === selectedMethodId
  ) ?? null;
}, [selectedProject, selectedMethodId]);
```

12.2. 遅延ロード

```
// クライアント側レンダリングフラグ（ハイドレーションエラー防止）
useEffect(() => {
```

```
  setIsClient(true);
}, []);
```

---

13. エラーハンドリング

13.1. 選択エラー

- プロジェクト未選択時: 選択ボタン・承認依頼ボタンを無効化
- メソッド未選択時: タスク選択を許可（'-'を選択可能）
- タスク未選択時: データ記録操作を制限

13.2. データ取得エラー

```
// プロジェクトデータが存在しない場合
const selectedProjectData = selectedProject
  ? projectsData[selectedProject]
  : null;

if (!selectedProjectData) {
  return <Empty description="プロジェクトデータが見つかりません" />;
}
```

---

14. 多言語化

14.1. 翻訳キー

```
const { t } = useLanguage();

// タブラベル
t('menu.experiment_worker.tabs.progressCheck')       // 進捗確認
t('menu.experiment_worker.tabs.workInstruction')     // 指示確認
t('menu.experiment_worker.tabs.experimentRecord')    // 実験記録
```

```
// ラベル
t('experimentWork.labels.project')                    // プロジェクト
t('experimentWork.selectButton')                      // 選択

// ステータス
t(`experimentWork.projectStatus.${project.status}`)  // 各種ステータス
```

---

15. 使用API

15.1. プロジェクト一覧取得API

エンドポイント: `GET /api/v1/projects/worker-accessible`

用途: 実験作業者がアクセス可能なプロジェクト選択コンボボックスの選択肢を取得

クエリパラメータ:

```
page: number (optional, default: 1) - ページ番号
limit: number (optional, default: 100) - 1ページあたりの件数
status: "1" | "2" | "3" (optional) - ステータスフィルタ（1: 作業中, 2: 承認待ち, 3: 承認済み）
sort_by: "create_at" | "project_name" | "update_at" (default: "project_name") - ソート項目
order: "asc" | "desc" (default: "asc") - ソート順
```

レスポンス:

```
{
  "success": true,
  "data": [
    {
      "projectId": "string (UUID)",
      "projectName": "string",
      "description": "string",
      "projectStatus": number,
      "useDeemedProgress": boolean,
      "createdAt": "string (ISO 8601)",
      "updatedAt": "string (ISO 8601)"
    }
  ],
  "pagination": {
    "page": number,
```

```
    "limit": number,
    "totalItems": number,
    "totalPages": number,
    "hasNext": boolean,
    "hasPrev": boolean
  },
  "meta": {
    "timestamp": "string (ISO 8601)",
    "requestId": "string (UUID)"
  }
}
```

レスポンス例:


```
{
  "success": true,
  "data": [
    {
      "projectId": "435b0667-af37-49e9-bc21-58f748587e49",
      "projectName": "GrapheneComposite",
      "description": "グラフェン複合材料の合成と物性評価",
      "projectStatus": 2,
      "useDeemedProgress": false,
      "createdAt": "1970-01-01T00:00:00.000Z",
      "updatedAt": "1970-01-01T00:00:00.000Z"
    },
    {
      "projectId": "9a92873d-9deb-471d-8ccf-bce2a2cd216b",
      "projectName": "HighTempSuperconductor",
      "description": "YBCO系高温超伝導体の合成条件最適化プロジェクト",
      "projectStatus": 3,
      "useDeemedProgress": false,
      "createdAt": "1970-01-01T00:00:00.000Z",
      "updatedAt": "1970-01-01T00:00:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 100,
    "totalItems": 4,
    "totalPages": 1,
    "hasNext": false,
    "hasPrev": false
  },
  "meta": {
    "timestamp": "2026-02-19T08:45:13.397Z",
    "requestId": "d6351ea7-b296-4867-b793-3a8858e46dac"
```

```
  }
}
```

使用箇所:

- プロジェクト選択コンボボックスの初期化時
- プロジェクト選択モーダル（`ProjectSelectionModal`）

実装状況: 実装済み（`getWorkerAccessibleProjects()` を使用）

特記事項:

- マルチテナント分離により自組織のプロジェクトのみ取得
- フロントエンドではフィルタリングせず、APIが返すすべてのプロジェクトを表示（`isSelectableStatus()` は使用しない）
- ステータスラベルは `useCodeMaster('project_status')` から取得
- 実験作業者に割り当てられたプロジェクトのみ表示する場合、将来的にクエリパラメータで `assignedTo={userId}` を追加

15.2. 承認依頼API

エンドポイント: `PATCH /api/v1/projects/{projectName}/approval-request`

用途: 進捗確認タブから承認依頼を送信

パスパラメータ:

- `projectName`: プロジェクト名（UUID ではなく名前文字列）

リクエストボディ:

```
{
  "comment": "string (optional)"
}
```

レスポンス:

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "timestamp": "string (ISO 8601)",
    "requestId": "string (UUID)"
  }
}
```

使用箇所:

- 進捗確認タブの承認依頼ボタン（`currentTab === '4'` の時のみ表示）

前提条件:

- プロジェクトが選択されている（`selectedProject !== ''`）
- ユーザーに承認依頼権限がある（実験作業者ロール）

実装状況: 実装済み（`projectApiService.ts` の `requestApproval(projectName)` を使用）

特記事項:

- パスパラメータは `projectId`（UUID）ではなく `projectName`（プロジェクト名の文字列）を使用
- リクエストボディには `comment` のみ（`requestedTasks` は不要）
- 承認依頼送信後、フロントエンドはプロジェクト一覧を再取得してステータスを反映

エラーケース:

- プロジェクトが選択されていない → ボタンを disabled にして防止
- 権限不足 → 403 Forbidden
- 既に承認依頼済み → 409 Conflict

15.3. API呼び出しフロー

プロジェクト一覧取得（マウント時）

```
// コンポーネントマウント時
useEffect(() => {
  const fetchProjects = async () => {
    setProjectsLoading(true);
    try {
      const result = await getWorkerAccessibleProjects();
      if (result.success) {
        setProjects(result.data);
      }
    } catch (error) {
      console.error('プロジェクト一覧取得エラー:', error);
      message.error('プロジェクト一覧の取得に失敗しました');
    } finally {
      setProjectsLoading(false);
    }
  };

  fetchProjects();
}, []);
```

承認依頼時

```
const handleApprovalRequest = async () => {
  if (!selectedProject || !selectedProjectData) return;

  setApprovalLoading(true);
  try {
    // projectName（プロジェクト名の文字列）で呼び出す
    await requestApproval(selectedProjectData.projectName);
    message.success('承認依頼を送信しました');

    // プロジェクト一覧を再取得してステータスをUIに反映
    const projectsResult = await getWorkerAccessibleProjects();
    if (projectsResult.success) {
      setProjects(projectsResult.data);
    }
  } catch (error) {
    message.error('承認依頼の送信に失敗しました');
  } finally {
    setApprovalLoading(false);
  }
};
```

---

16. テスト観点

16.1. ユニットテスト

- [ ] プロジェクト変更時にメソッド・タスクがリセットされる
- [ ] メソッド変更時にタスクがリセットされる
- [ ] タブ切り替えが正常に動作する
- [ ] 承認依頼ボタンが進捗確認タブのみで表示される
- [ ] 選択ボタンがプロジェクト選択時に有効になる

16.2. 統合テスト

- [ ] プロジェクト選択モーダルから選択後、状態が正しく反映される
- [ ] メソッド・タスク選択モーダルから選択後、状態が正しく反映される
- [ ] タブ間でプロジェクト・メソッド・タスクの状態が共有される

16.3. E2Eテスト

- [ ] ログイン後、実験作業者メニューが表示される
- [ ] プロジェクトを選択し、各タブを切り替えられる

16.4. API連携テスト

- [ ] プロジェクト一覧取得APIが正常に動作する
- [ ] 選択可能なステータスのプロジェクトのみ表示される
- [ ] 承認依頼APIが正常に動作する
- [ ] 承認依頼後、プロジェクトステータスが更新される
- [ ] API エラー時に適切なエラーメッセージが表示される